Project Report

Hidden Treasures is a two-player game developed in Python with a graphical interface built using `breezypythongui`. The gameplay is based on successive rounds in which players build a 5-card hand, managing an Action Points budget and exploiting three special cards with strategic effects.

1. Adopted Strategy

System Objective

The application must allow a complete two-player game on a 6×6 grid, with turn-taking, action constraints, adapted poker-style score calculation, and persistent leaderboard saving.

Logical Architecture

The solution is organized according to the MVC (Model-View-Controller) pattern, separating the domain logic (Model) from the interface management (View/Controller).

Data Model

Represents the fundamental objects of the domain, namely Card, Deck, and Player. These classes encapsulate state and basic operations such as covering, shuffling, hand management, and action points.

Evaluation Engine

A module dedicated to hand evaluation with scoring rules and the management of the Gem as a constrained wildcard. The validator provides check methods and a scoring function that returns the round score as the sum of the combination and the remaining action points.

Interface and Flow Control

The GUI manages the button grid, clicks, actions, turn alternation, timer, round progression, and game end. It also applies the effects of the Coin and Scroll special cards and coordinates the visual state update, including the graphic evidence of card possession in the grid.

Persistence and Leaderboard

At the end of the game, the result is appended to a text file. The leaderboard is reconstructed through parsing and sorting, respecting the required rules: descending winner score, then score difference, then alphabetical order of the winner.

2. Functional Description of the Game

Setup of a Round

At the beginning of each round, a 6×6 grid is built with 36 cards drawn from a larger deck. The cards are initially face down and are revealed only following the action of the active player. Each player starts with 15 Action Points and must complete a 5-card hand.

Game Turn and Action Constraints

During the turn, a player selects a face-down card on the grid to reveal it temporarily, then chooses one of the allowed actions:

- **Accept**: Costs 1 Action Point, adds the revealed card to the hand only if the hand is not full.
- **Reject**: Costs 1 Action Point, the card returns face down and is not assigned. It is allowed only if the remaining action points are strictly greater than the number of missing cards to complete the hand.
- **Change**: Costs 2 Action Points, replaces a card in the hand with the revealed card, choosing which one to remove. It is allowed only if there is at least one card in the hand and if the remaining action points respect the required constraint after the action.

The GUI implements these constraints by dynamically enabling and disabling action buttons based on the current state of the turn, the hand, and the action points.

Special Cards and Their Effects

Special cards do not belong to any suit and introduce specific mechanics.

- **Gem**: Functions as a wildcard only for combinations based on equal values, thus pair, two pair, three of a kind, full house, four of a kind. It cannot complete a straight, flush, or straight flush. The hand containing the Gem is evaluated by choosing the value that maximizes the admissible score.
- **Coin**: When accepted, immediately assigns an additional action point. It does not participate in combinations but can indirectly increase the final score, since the remaining action points are added to the round score.
- **Scroll**: When accepted, allows permanently revealing a face-down card of choice on the grid. The card remains visible for the rest of the round and is not automatically assigned, thus remaining strategic information.

Conclude and End of the Round

When a player completes their hand, they can choose to conclude their participation in the round to conserve their remaining action points. The round ends when both players have concluded or when both have run out of action points. At the end of the round, each player's score is calculated by adding the combination score to the remaining action points. The next round starts by reversing who started in the previous round, so as to balance the advantage of the first turn.

Game Over and Victory Criteria

Round scores accumulate and the game ends when at least one player reaches 110 points. If both reach the threshold in the same round, the one with the higher score wins, then in case of a tie, the one with more remaining action points, then a possible draw.

3. Game Audio Management and Motivations for the Technological Choice

Functional Introduction

To improve the user experience and make the gameplay more immersive, the project integrates an audio component in the form of background music played in a loop during the entire gaming session. The music has no direct ludic function, but contributes to creating continuity, rhythm, and emotional involvement, fundamental elements in interactive video game applications.

Library Choice

For audio management, the `pygame` library was used, limited to its sound reproduction module. The choice was guided by criteria of simplicity, reliability, and compatibility with the project architecture. In particular, `pygame` allows to:

- Play mp3 audio files directly
- Easily manage continuous loop playback
- Start and maintain background audio without interfering with the main GUI loop
- Operate independently from the game logic and the graphical event system

The use of `pygame` was restricted exclusively to audio management, avoiding superfluous dependencies or unnecessary uses of the library. In this way, the sound component remains modularized and does not introduce additional complexity into the overall software structure.

Integration with the Graphical Interface

Music playback is initialized only once when the application starts and remains active for the duration of the game. This design choice avoids repeated or redundant calls to the audio system during player turns or actions, ensuring stability and sonic continuity.

From an architectural point of view, the music is not tied to single screens or events, but is treated as a global game resource. This approach respects the principle of separation of responsibilities, since the audio logic does not interfere with turn management, scoring, or user interaction.

Impact on Performance

From the point of view of algorithmic complexity, audio management has a constant cost $O(1)$. Loop playback does not involve computational cycles dependent on the size of the game data and does not affect the performance of the GUI or the responsiveness of the interface. The initialization of the audio system occurs only once and subsequent operations are delegated to the library's internal engine, making the computational impact negligible compared to the operations of updating the grid or evaluating game hands.

Design Coherence

The integration of audio via `pygame` is consistent with the project's objective, which is to create a complete, interactive, and usable game. The technological choice favors an established and lightweight solution, keeping the code readable, separated by responsibilities, and easily extendable in future developments, such as the addition of sound effects linked to specific game actions.

4. Main Classes and Methods

4.1 Card

The `Card` class represents both numerical cards with value and suit, and special cards with `specialType`. It also manages the visibility state.

- **Main Attributes**: `value`, `suit`, `specialType`, `faceDown`, `permanentlyRevealed`.
- **__str__**: Provides a textual representation of the card, distinguishing special and numerical ones, with management of face cards only for textual rendering. This method is useful for debugging and for any textual elements in the GUI.
- **flip_card**: Inverts the state (from face down to face up and vice versa), but blocks the operation if the card was permanently revealed via a scroll.
- **reveal_permanently**: Sets `permanentlyRevealed` and uncovers the card, making the scroll choice persistent for the round.
- **reset**: Restores the initial state. It is used when a card is removed from the hand or when a new round starts.

Complexity: All operations of the `Card` class are $O(1)$, because they act on a constant number of attributes.

4.2 Deck

The `Deck` class builds the set of cards, shuffles, and provides the 36 cards for the 6×6 grid.

- **createDeck**: Generates numerical cards for each suit and adds special cards, then shuffles.
- **shuffle**: Random shuffling of the list via `shuffle`.
- **draw_36**: Returns the first 36 cards and removes them from the deck, to populate the round grid.

Complexity:

- `createDeck` is $O(N)$ where N is the number of generated cards, because it creates all instances and then shuffles.
- `shuffle` is $O(N)$ in the average case, consistent with the list shuffle algorithm.
- `draw_36` is $O(36)$ for logical extraction, which is practically constant, and the slicing operation is proportional to the extracted size.

4.3 Player

The `Player` class encapsulates name, hand, action points, and round completion state.

- **add_card**: Adds a card to the hand if it is not full, i.e., maximum 5 cards.
- **remove_card**: Removes a card from the hand and invokes `reset` on the removed card, so as to restore its state.
- **reset_round**: Resets the hand, restores 15 action points, and sets concluded to false.

Complexity:

- `add_card` is $O(1)$
- `remove_card` is $O(K)$ where K is the hand size, at most 5, therefore essentially constant
- `reset_round` is $O(K)$ with K at most 5

4.4 Validator

This class implements the evaluation rules of adapted poker and the round score calculation, including the management of the Gem.

- **SCORES**: Maps combination to fixed score, with royal flush 100, four of a kind 50, full house 30, flush 25, straight 20, three of a kind 15, two pair 10, pair 5.
- **evaluateHand(cards, remainingActionPoints)**: Calculates the combination score and adds the remaining action points, as required by the assignment.

- **calculateComboScore**: Evaluates straights and flushes separately, then groups of equal values, and takes the maximum between the two branches, ensuring that the best valid combination determines the combo score.
- **manageGem**: If the hand contains the Gem, treats it as a suitless wildcard that can assume a numerical value to improve only grouped combinations. Implements a search on the value to assign based on occurrences, choosing the outcome with the maximum score among four of a kind, full house, three of a kind, two pair, pair.
- **checkRoyalFlush, checkFourOfAKind, checkFullHouse, etc.**: Control methods that verify combination conditions, using value counts and checks on suits and consecutiveness.

Complexity: For both game constraints and design choices, the hand has a maximum cardinality of 5, so the actual costs are constant. In general asymptotic terms:

- Value extraction, counting, and `Counter`-based checks are $O(K)$
- Value sorting for the straight is $O(K \log K)$
- `manageGem` iterates over the distinct values present in the hand, at most K, so $O(K)$ with constant internal operations.

With K fixed at 5, execution is always very efficient and suitable for frequent updates in the GUI, even during the turn.

4.5 Game and GUI

The main GUI class manages the global state of the game, the grid, action buttons, timer, and turn alternation.

- **manageTurn**: Updates the information panel and enables/disables controls consistently with action point and hand size constraints. Also updates a provisional score as the sum of total points and the current evaluation of the hand with remaining action points.
- **changeTurn**: Alternates the active player, handles cases where a player has concluded or run out of action points, and updates the grid showing assigned cards only to the owner, while others remain face down, except for permanent scroll revelations.
- **revealCard**: Handles clicking on the grid. Includes two important sub-dynamics:
  - *Scroll mode*, where an unassigned card is permanently revealed
  - *Exchange mode*, where the card to replace in the hand is selected during the Change action
- **actionConclude**: Allows closing the round for the player when the hand is complete, as required by the assignment.
- **endRound**: Temporarily stops the timer, calculates and accumulates player scores by calling the Validator, resets the grid, and prepares the next round if no one has reached 110 points.

- **endGame**: Applies victory and tie-breaker rules, determining the winner or a draw consistently with the assignment.

Complexity: Many GUI operations iterate over the grid, so they have cost $O(G)$ with G equal to 36, which is constant in the game. Turn updating, including managing images and button states, is therefore efficient and compatible with frequent updates.

Application Startup, Main Menu, and Leaderboard

Role of the `main.py` class in the project

The `main.py` file represents the entry point of the entire application. Its function is to initialize the graphical environment, manage preliminary and service screens, and start the actual game. From an architectural perspective, this class does not implement game logic or scoring algorithms, but acts as a high-level controller, dealing with:

- Presenting the main menu
- Managing background music playback
- Starting the game window
- Displaying the persistent leaderboard
- Coordinating the transition between different application windows

This approach allows separating startup and navigation logic from internal game logic, improving code readability, modularity, and maintainability.

Integration of Tkinter and limitations of BreezyPython GUI in Leaderboard management

To create the leaderboard screen, it was necessary to integrate the direct use of the standard `tkinter` library, alongside the `breezypythongui` wrapper used for the game core. The choice is motivated by the aesthetic and functional customization constraints imposed by the wrapper, which abstracts widget management by simplifying its grid layout but drastically reducing control over low-level rendering parameters. Specifically, the leaderboard screen required prerequisites not satisfiable via standard `breezypythongui` classes:

- **Layering of elements**: The need to position a scrollable `Listbox` on top of a custom background image required the use of the `Canvas` widget and the `create_window` method, features natively managed by `tkinter` but not clearly exposed by the wrapper.
- **Advanced Widget Styling**: To achieve a visual rendering consistent with the game theme, it was necessary to access specific `Listbox` widget parameters such as `borderwidth` and `highlightthickness`. Setting these values to zero, crucial for removing default borders and the OS focus rectangle, is not transparently supported by `breezypythongui`, which tends to force predefined styles.

Adopting a hybrid approach allowed maintaining development speed for the game grid (where `breezypythongui` is efficient) while ensuring the necessary flexibility to refine the visual experience in menu and leaderboard screens.

Main Menu Class

Main Responsibilities The `MainMenu` class, derived from `EasyFrame`, represents the application's initial screen. It has the task of offering a clear and immediate access point to the game, combining graphical elements, user interaction, and audio components. Specifically, the class manages:

- Opening the application in full screen
- Loading and dynamically resizing the cover image
- Initializing and playing background music in a loop
- Displaying the game start button

Initialization and graphical layout During initialization, the window automatically adapts to the user's screen resolution. The cover image is resized while maintaining high visual quality, ensuring uniform rendering on devices with different resolutions. Using a canvas allows overlaying interactive elements on the background image, enabling flexible and centered positioning of the start button relative to the screen.

Audio Management Upon menu startup, background music is initialized and played in a continuous loop. The audio component is independent of the game logic and is started only once, avoiding duplications or interference with the graphical event loop. If audio playback is unavailable, the application continues to function, preserving startup robustness.

Game Start The `start_game` method handles the transition from the menu to the main game window. Pressing the "Play" button closes the menu window and instantiates the class managing the actual game. This separation ensures that the menu remains an independent screen and does not interfere with turn, grid, or score management.

Computational Complexity Operations performed by `MainMenu` have constant computational cost:

- Image loading and audio initialization are executed only once
- Button and click event management is $O(1)$
- There are no loops dependent on the size of the game data

Therefore, the algorithmic complexity of the class is $O(1)$ relative to the problem size, with a negligible impact on the application's overall performance.

Leaderboard Class

Function of the leaderboard screen The `Leaderboard` class, also derived from `EasyFrame`, is dedicated to displaying the results of previous games saved persistently. It provides a separate screen, accessible from the interface, allowing users to consult the leaderboard in textual form, ordered according to the rules defined in the data management module.

Loading and displaying data The class:

- Loads a dedicated graphical background
- Reads leaderboard data from a text file via the `LeaderboardManager` class

- Inserts formatted rows into a `Listbox`

Choosing a `Listbox` allows for an orderly, readable, and easily scrollable display of results, maintaining a style consistent with the game's aesthetics.

Navigation A button allows returning to the main menu, closing the leaderboard window, and reopening the initial screen. This mechanism ensures clear and linear navigation between the application's screens.

Computational Complexity Both loading the background and creating the graphical interface have a constant cost. Reading and displaying the leaderboard depend on the number of saved rows, denoted by R.

- Reading and parsing data: $O(R)$
- Inserting rows into the `Listbox`: $O(R)$

The overall complexity of the leaderboard screen is therefore $O(R)$, making it adequate and scalable concerning the number of saved games.

Application Startup

The final block of the `main.py` file starts the application by instantiating the `MainMenu` class and starting the graphical interface's main loop. This makes `main.py` the formal entry point of the project, complying with standard conventions for Python applications featuring a graphical interface.

4.6 Leaderboard Manager

The `LeaderboardManager` class handles the persistence of game data on the file system. In addition to I/O operations, this module is responsible for data normalization and validation to ensure leaderboard consistency even with older save formats or tie scenarios.

- **Attributes and Constants**: `TIE_VALUE`: String constant ("DRAW") used as a standardized value in the winner field when there is no single triumphant player.
- **Game**: Represents the record of a single game. Includes static methods for string conversion and data cleaning.
- **Normalization and Backward Compatibility**: The class implements robust logic for handling the winner field via the `normalize_winner` static method. This method verifies that the winner's name matches one of the two players; otherwise (or if the field is empty/null), the value is forced to the `TIE_VALUE` constant. The row parsing (`from_line`) also handles backward compatibility: if the file contains rows generated by older software versions (with 9 fields instead of 10), the system automatically accepts them by assigning the default tie value, preventing read errors.
- **record_game**: Method to append data to the `leaderboard.txt` file. Before saving, it invokes winner normalization to ensure no "dirty" or inconsistent data is written to disk. It saves ISO timestamp, duration, game metrics, and winner, separated by pipes (`|`).
- **leaderboard_as_text**: The method reconstructs statistics by reading the entire history. For each player, it calculates:

- *Victories*: Incremented only if the normalized winner field strictly matches the player's name. Tied games are counted in the total games played but do not increment the victory counter for either contender.
- *Statistics*: Calculates best score, average points, and average game duration.

The final sorting uses a stable multi-key criterion:

1. Number of Victories (descending).
2. Best Score (descending).
3. Average Score (descending).
4. Name (ascending alphabetical).

Complexity: Operations maintain high efficiency: $O(1)$ for normalized writing and $O(R)$ for reading and adapting legacy rows, where R is the number of historical games.

5. Persistence and Leaderboard Sorting

Saving

Games are saved to a text file in append mode. Each row represents a single completed game and contains information such as date, game duration, number of rounds, player names, final scores, remaining action points, and the winner's name. Fields are separated by a delimiter character, enabling simple and deterministic parsing during reading. This solution guarantees lightweight persistence, easily inspectable and independent of external database systems.

Sorting

The leaderboard is sorted according to the criteria defined by the assignment:

- Winner's score in descending order
- In case of a tied score, difference between the two players' scores in descending order
- In case of a further tie, ascending alphabetical order of the winner's name

Sorting is applied in memory to the data read from the file, prior to user display.

Adopted Sorting Algorithm

For leaderboard sorting, the Insertion Sort algorithm was explicitly implemented. This is a simple comparison sorting algorithm that progressively builds the sorted sequence by inserting each new element into its correct position relative to the already processed ones. The choice of Insertion Sort is consistent with the project's application context, because:

- The number of saved results is limited and grows slowly
- The algorithm is simple to implement and verify

- Its behavior is deterministic and easily controllable
- It allows clear application of a multi-level sorting criterion

Multi-key sorting is managed directly during the comparison between two elements, sequentially evaluating the winner's score, the score difference, and finally the name, ensuring strict adherence to the leaderboard rules.

Computational Complexity Assuming R saved results:

- Reading and parsing the file have complexity $O(R)$
- The Insertion Sort algorithm has complexity $O(R^2)$ in the worst case

In the application's context, the value of R remains contained, making the computational cost of sorting fully acceptable. The algorithm's simplicity and the sorting process's clarity are therefore more relevant than asymptotic scalability, without perceivable impacts on performance or user experience.

6. Single-Player Mode and Heuristic Artificial Intelligence

Functional Overview

To expand the gameplay capabilities of the "Hidden Treasures" application, a Single-Player mode has been introduced, allowing a human user to compete against an automated opponent (Player 2). Given that the core mechanics revolve around managing a strict Action Points budget and exploiting special cards with strategic effects, the automated agent is governed by a heuristic state machine algorithm. This approach ensures deterministic, probabilistically sound decision-making without introducing the significant computational overhead or external library dependencies associated with deep reinforcement learning models.

Heuristic Algorithm Behavior

The bot's artificial intelligence dynamically evaluates the current game state, the composition of its hand, and the remaining Action Points to optimize its moves according to predefined logical constraints:

- **Base Rules and Resource Management**: During the initial phases of a round, the bot prioritizes the "Accept" action to establish a playable baseline. The algorithm strictly regulates the "Reject" action, executing it only if the remaining Action Points budget maintains a calculated safety margin relative to the number of cards required to complete the 5-card hand.
- **The Coin (Mathematical Optimization)**: The bot is programmed to instantly accept the Coin card upon discovery. The algorithm recognizes the absolute mathematical advantage of acquiring an additional Action Point at a net-zero operational cost.

- **The Gem (Strategic Pivot)**: The Gem card acts as a critical pivot point within the state machine. Due to its function as a constrained wildcard for combinations based on equal values, its acquisition is elevated to the highest priority. If the bot's hand is full, the algorithm forces a "Change" action, discarding the card with the lowest statistical potential. Once the Gem is integrated, the bot executes a strategic pivot: it permanently aborts any heuristics aimed at constructing Straights or Flushes—which the Gem cannot complete—and exclusively targets cards of identical numerical value to maximize the probability of achieving high-scoring groupings (e.g., Three of a Kind, Full House, or Four of a Kind).
- **The Scroll (Information Memory)**: The bot incorporates a short-term memory mechanism triggered by the Scroll card. Provided a sufficient Action Point budget (e.g., AP > 8), the bot accepts the Scroll to permanently reveal a face-down card on the grid. In subsequent turns, prior to executing random coordinate selections, the algorithm queries this memorized card; if its numerical value or suit improves the current hand composition, the bot deterministically targets that specific coordinate.

GUI Integration and Asynchronous Execution

To maintain a seamless and accessible user experience, the bot's decision loop is integrated into the main graphical interface using asynchronous timers. Rather than executing its entire turn instantaneously, the algorithm introduces a progressive, artificial delay before each action. This non-blocking pacing prevents the interface from freezing, preserving the interactive flow of the application.